# Nested Loops in C

In the programming context, the term "nesting" refers to enclosing a particular programming element inside another similar element. For example, nested loops, nested structures, nested conditional statements, etc.

## Nested Loops

When a looping construct in C is employed inside the body of another loop, we call it a **nested loop** (or, loops within a loop). Where, the loop that encloses the other loop is called the **outer loop**. The one that is enclosed is called the **inner loop**.

## General Syntax of Nested Loops

The general form of a nested loop is as follows −

```
Outer loop {
   Inner loop {
      ...
      ...
   }
   ...
}
```

C provides three keywords for loops formation − while, do-while, and for. Nesting can be done on any of these three types of loops. That means you can put a **while** loop inside a **for** loop, a **for** loop inside a **do-while** loop, or any other combination.

The general behaviour of nested loops is that, for each iteration of the outer loop, the inner loop completes all the iterations.

## Nested For Loops

**Nested for loops** are very common. If both the outer and inner loops are expected to perform three iterations each, the total number of iterations of the innermost statement will be "3 * 3 = 9".

## Example: Nested for Loop

Take a look at the following example −

</> Open Compiler

```c
#include <stdio.h>

int main(){

   int i, j;

   // outer loop
   for(i = 1; i <= 3; i++){

      // inner loop
      for(j = 1; j <= 3; j++){
         printf("i: %d j: %d\n", i, j);
      }
      printf("End of Inner Loop \n");
   }
   printf("End of Outer Loop");

   return 0;
}
```

## Output

When you run this code, it will produce the following output –

```
i: 1 j: 1
i: 1 j: 2
i: 1 j: 3
End of Inner Loop

i: 2 j: 1
i: 2 j: 2
i: 2 j: 3
End of Inner Loop

i: 3 j: 1
i: 3 j: 2
i: 3 j: 3
End of Inner Loop
```

```
End of Outer loop
```

## Explanation of Nested Loop

Now let's analyze how the above program works. As the outer loop is encountered, "i" which is the looping variable for the outer loop is initialized to 1. Since the test condition (a <= 3) is true, the program enters the outer loop body.

The program reaches the inner loop, and "j" which is the variable that controls the inner loop is initialized to 1. Since the test condition of the inner loop (j <= 3) is true, the program enters the inner loop. The values of "a" and "b" are printed.

The program reaches the end of the inner loop. Its variable "j" is incremented. The control jumps to step 4 until the condition (j <= 3) is true.

As the test condition becomes false (because "j" becomes 4), the control comes out of the inner loop. The end of the outer loop is encountered. The variable "i" that controls the outer variable is incremented and the control jumps to step 3. Since it is the start of the inner loop, "j" is again set to 1.

The inner loop completes its iteration and ends again. Steps 4 to 8 will be repeated until the test condition of the outer loop (i <= 3) becomes false. At the end of the outer loop, "i" and "j" have become 4 and 4 respectively.

The result shows that, for each value of the outer looping variable, the inner looping variable takes all the values. The total lines printed are "3 * 3 = 9".



## Nesting a While Loop Inside a For Loop

Any type of loop can be nested inside any other type. Let us rewrite the above example by putting a **while** loop inside the outer **for** loop.

## Example: Nested Loops (while Loop Inside for Loop)

Take a look at the following example –

</> Open Compiler

```c
#include <stdio.h>

int main(){
```

```
   int i, j;

   // outer for loop
   for (i = 1; i <= 3; i++){

      // inner while loop
      j = 1;
      while (j <= 3){
         printf("i: %d j: %d\n", i, j);
         j++;
      }
      printf("End of Inner While Loop \n");
   }
   printf("End of Outer For loop");

   return 0;
}
```

## Output

```
i: 1 j: 1
i: 1 j: 2
i: 1 j: 3
End of Inner While Loop

i: 2 j: 1
i: 2 j: 2
i: 2 j: 3
End of Inner While Loop

i: 3 j: 1
i: 3 j: 2
i: 3 j: 3
End of inner while Loop

End of outer for loop
```

Programmers use nested loops in a lot of applications. Let us take a look at some more examples of nested loops.

# C Nested Loops Examples

## Example: Printing Tables

The following program prints the tables of 1 to 10 with the help of two nested **for** loops.

</> Open Compiler

```
#include <stdio.h>

int main(){

   int i, j;
   printf("Program to Print the Tables of 1 to 10 \n");

   // outer loop
   for(i = 1; i <= 10; i++){

      // inner loop
      for(j = 1; j <= 10; j++){
         printf("%4d", i*j);
      }
      printf("\n");
   }
   return 0;
}
```

## Output

Run the code and check its output –

```
Program to Print the Tables of 1 to 10
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
   4   8  12  16  20  24  28  32  36  40
   5  10  15  20  25  30  35  40  45  50
   6  12  18  24  30  36  42  48  54  60
   7  14  21  28  35  42  49  56  63  70
   8  16  24  32  40  48  56  64  72  80
```

```
  9  18  27  36  45  54  63  72  81  90
 10  20  30  40  50  60  70  80  90 100
```

## Example: Printing Characters Pyramid

The following code prints the increasing number of characters from a string.

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main(){

   int i, j, l;
   char x[] = "TutorialsPoint";
   l = strlen(x);

   // outer loop
   for(i = 0; i < l; i++){

      // inner loop
      for(j = 0; j <= i; j++){
         printf("%c", x[j]);
      }
      printf("\n");
   }
   return 0;
}
```

## Output

When you run this code, it will produce the following output –

```
T
Tu
Tut
Tuto
Tutor
Tutori
```

```
Tutoria
Tutorial
Tutorials
TutorialsP
TutorialsPo
TutorialsPoi
TutorialsPoin
TutorialsPoint
```

## Example: Printing Two-Dimensional Array

In this program, we will show how you can use **nested loops** to display a two-dimensional array of integers. The outer loop controls the row number and the inner loop controls the columns.

</> Open Compiler

```c
#include <stdio.h>

int main(){

   int i, j;
   int x[4][4] = {
      {1, 2, 3, 4},
      {11, 22, 33, 44},
      {9, 99, 999, 9999},
      {10, 20, 30, 40}
   };

   // outer loop
   for (i=0; i<=3; i++){

      // inner loop
      for(j=0; j <= 3; j++){
         printf("%5d", x[i][j]);
      }
      printf("\n");
   }
   return 0;
}
```

## Output

When you run this code, it will produce the following output –

```
 1   2   3   4
11  22  33  44
 9  99 999 9999
10  20  30  40
```